

App de Caronas

Ricardo Rocha
Aaron Guizoli
Italo Avelar
Igor Hendrix

App de Caronas

===== App de Caronas UFMG =====

Alunos: Ricardo Rocha, Aaron Guizoli, Italo Avelar e Igor Hendrix

Apresentação do Problema:

A comunidade acadêmica da UFMG, composta por milhares de alunos, professores e funcionários, enfrenta desafios diários de mobilidade. A grande extensão do campus Pampulha, somada à dispersão dos membros da comunidade por diversas regiões de Belo Horizonte, resulta em dificuldades de locomoção. A falta de uma plataforma centralizada e segura para a organização de caronas solidárias impede que motoristas com vagas ociosas se conectem de forma eficiente com passageiros que fazem rotas similares.

O problema central é a ausência de um sistema confiável que facilite a coordenação de caronas, verifique a identidade dos participantes através do vínculo com a universidade e ofereça um ambiente seguro, especialmente para públicos que demandam maior segurança, como as mulheres.

Objetivo:

Implementar uma plataforma de caronas solidárias em C++ para a comunidade da UFMG, com o objetivo de otimizar a mobilidade e fortalecer a segurança dos usuários. O sistema conecta motoristas e passageiros de forma eficiente, garantindo a identidade dos participantes através da validação institucional e fomentando uma comunidade confiável por meio de um sistema de reputação.

Visão Geral da Solução:

Este projeto, o "App de Caronas UFMG", é uma aplicação que simula um sistema completo de caronas solidárias, projetado para atender às necessidades da comunidade da UFMG. A solução foca em segurança, usabilidade e uma arquitetura robusta e modular.

O sistema permite que usuários, após validarem seu CPF em uma base de dados simulada da UFMG, se cadastrem na plataforma. Uma vez cadastrados, eles podem:

- Como Passageiro: Buscar caronas disponíveis com base em zonas de origem e destino, solicitar participação, negociar pontos de embarque e avaliar os motoristas após a viagem.
- Como Motorista: Cadastrar seus veículos, oferecer caronas (agendadas ou imediatas), gerenciar solicitações de passageiros, e avaliar os passageiros após a viagem.

A segurança é um pilar central, com funcionalidades como a opção de caronas exclusivas para mulheres e um sistema de avaliação mútua que constrói uma reputação para cada usuário, incentivando um comportamento cordial e seguro.

Estrutura e Funcionamento do Programa:

O projeto foi desenvolvido em C++11, seguindo os princípios da Programação Orientada a Objetos para garantir modularidade, manutenibilidade e escalabilidade. A arquitetura é dividida em classes de entidade, classes de serviço (gerenciadores) e uma classe orquestradora principal.

Principais Estruturas de Dados e Classes:

Entidades:

- Usuario: Classe base polimórfica que contém dados comuns (nome, cpf, etc.). Motorista: Classe derivada de Usuario, com funcionalidades adicionais como CNH e uma lista de Veiculo.

- Veiculo: Armazena dados de um veículo (placa, modelo, cor, lugares).
- Carona: A entidade central, que agrega informações de rota, horário, motorista, passageiros e status.
- Solicitudao: Modela o processo de um passageiro pedindo para entrar em uma carona, incluindo o status da negociação.
- Avaliacao: Representa a nota e o comentário que um usuário faz a outro.
- Notificacao: Mensagens simples enviadas aos usuários.

Gerenciadores (Classes de Serviço): A lógica de negócio é encapsulada em classes "Gerenciador", cada uma com uma responsabilidade única:

- GerenciadorUsuarios: Controla cadastro, login e perfis.
- GerenciadorVeiculos: Gerencia os veículos de um motorista.
- GerenciadorCaronas: Orquestra a criação e o ciclo de vida das caronas.
- GerenciadorSolicitacoes: Media a interação entre passageiros e caronas.
- GerenciadorAvaliacoes: Controla o processo de avaliação mútua.

Orquestração e Interface:

- Sistema: A classe principal que inicializa todos os subsistemas, gerencia o estado da aplicação (ex: usuário logado) e contém o loop principal do programa.
- TerminalIO: Uma classe de serviço que abstrai toda a interação com o console, responsável por exibir menus e coletar dados do usuário.
- Utilitarios: Fornece funções auxiliares, como manipulação de datas e conversão de enums para strings.

Funcionamento:

1. A main.cpp instancia e executa a classe Sistema.
2. O construtor de Sistema inicializa todos os gerenciadores, que por sua vez carregam os dados de arquivos de texto (simulando um banco de dados).
3. O Sistema entra em um loop, exibindo o menu apropriado (logado ou não logado).
4. As ações do usuário são delegadas aos gerenciadores correspondentes, que manipulam as entidades e a lógica de negócio.
5. Ao encerrar, o destrutor de Sistema garante que todos os dados sejam salvos de volta nos arquivos de texto.

Principais Dificuldades Encontradas:

- Gerenciamento de Memória e Propriedade (Ownership): A decisão de usar ponteiros brutos para representar associações entre objetos (ex: Carona apontando para Usuario) exigiu um design cuidadoso para evitar memory leaks e dangling pointers. Foi crucial definir claramente qual classe "gerenciador" seria a proprietária (owner) de cada tipo de objeto e responsável por sua desalocação, documentando isso rigorosamente.
- Manutenção do Estado e Polimorfismo: Converter um Usuario em Motorista após o login foi complexo. A solução adotada foi criar uma nova instância de Motorista, copiar os dados do Usuario original, e então deletar o objeto antigo, atualizando o ponteiro do usuário logado no Sistema. Isso exigiu cuidado para manter a consistência dos dados e gerenciar a memória corretamente.

Extras Implementados Além dos requisitos básicos, as seguintes funcionalidades foram implementadas para tornar o sistema mais robusto e completo:

- Sistema de Reputação ("Medalhas"): Foi criado um sistema de reputação visual. Com base na média de avaliações recebidas, os usuários recebem "medalhas" (Bronze, Prata, Ouro), que são exibidas em seus perfis, incentivando a boa conduta na plataforma.
- Persistência de Dados em Arquivos: Todo o estado do sistema (usuários, veículos, caronas, etc.) é salvo em arquivos de texto ao final da execução e recarregado no início. Isso simula um banco de dados e torna o estado da aplicação persistente entre as sessões.
- Fluxo de Negociação de Carona: Em vez de um simples "aceitar/recusar", o sistema permite que o motorista faça uma contraproposta de locais de embarque e desembarque, que o passageiro pode então aceitar ou recusar, tornando o processo de solicitação mais flexível e realista.

Índice da hierarquia

Hierarquia de classes

Esta lista de heranças está organizada, dentro do possível, por ordem alfabética:

ufmg_carona::Avaliacao	10
ufmg_carona::Carona	13
ufmg_carona::CaronaFactory	20
ufmg_carona::Notificacao	29
std::runtime_error	
ufmg_carona::AppExcecao	8
ufmg_carona::AutenticacaoFalhouException	9
ufmg_carona::CaronaLotadaException	22
ufmg_carona::ComandoInvalidoException	23
ufmg_carona::Sistema	31
ufmg_carona::Solicitacao	36
ufmg_carona::Usuario	39
ufmg_carona::Motorista	24
ufmg_carona::Veiculo	43

Índice dos componentes

Lista de componentes

Lista de classes, estruturas, uniões e interfaces com uma breve descrição:

ufmg_carona::AppExcecao (Classe base para todas as exceções específicas da aplicação)	8
ufmg_carona::AutenticacaoFalhouException (Exceção lançada quando uma tentativa de login falha)	9
ufmg_carona::Avaliacao (Modela uma avaliação que um usuário faz a outro após uma carona. [cite: 18])	10
ufmg_carona::Carona (A entidade central do sistema, agregando todas as informações de uma carona)	13
ufmg_carona::CaronaFactory (Implementa o padrão de projeto "Factory Method" para a criação de objetos Carona)	20
ufmg_carona::CaronaLotadaException (Exceção lançada ao tentar solicitar participação em uma carona sem vagas)	22
ufmg_carona::ComandoInvalidoException (Exceção lançada quando o sistema recebe um comando de usuário não reconhecido)	23
ufmg_carona::Motorista (Representa um usuário que também pode atuar como motorista, oferecendo caronas)	24
ufmg_carona::Notificacao (Encapsula uma mensagem e seu estado (lida/não lida) para comunicação com o usuário. Esta classe é usada para informar os usuários sobre eventos importantes, como a aceitação de uma solicitação de carona ou uma nova avaliação recebida)	29
ufmg_carona::Sistema (A classe principal que gerencia o estado, os dados e os fluxos de interação do usuário. Atua como um orquestrador para toda a aplicação, inicializando subsistemas, controlando o loop principal e garantindo a persistência dos dados)	31
ufmg_carona::Solicitacao (Modela uma solicitação de um passageiro para participar de uma carona. Esta classe encapsula todo o fluxo de negociação, incluindo propostas de locais e o estado final da solicitação, além de rastrear o status das avaliações pós-viagem)	36
ufmg_carona::Usuario (Classe base que encapsula os dados e comportamentos comuns a todos os usuários. Serve como a fundação para o polimorfismo no sistema, permitindo que diferentes tipos de usuários (como Motorista) sejam tratados de forma uniforme)	39
ufmg_carona::Veiculo (Modela um veículo, contendo todas as suas informações relevantes. Esta classe atua como uma estrutura de dados para armazenar os detalhes de um veículo que pode ser utilizado em uma carona)	43

Índice dos ficheiros

Lista de ficheiros

Lista de todos os ficheiros documentados com uma breve descrição:

include/Avaliacao.hpp (Define a classe Avaliacao, que encapsula os dados de uma avaliação feita por um usuário a outro no contexto de uma carona específica)	45
include/Carona.hpp (Define a classe Carona, a entidade central do sistema que representa uma viagem oferecida)	47
include/CaronaFactory.hpp	51
include/Excecoes.hpp (Define a hierarquia de exceções customizadas para a aplicação de caronas. Centralizar as exceções em um único arquivo promove um tratamento de erros consistente e padronizado em todo o sistema)	52
include/Genero.hpp (Define o enumerador Genero para representar as opções de identidade de gênero dos usuários)	54
include/Motorista.hpp (Define a classe Motorista, uma especialização da classe Usuario que representa um motorista no sistema)	56
include/Notificacao.hpp (Define a classe Notificacao, uma entidade simples para representar mensagens enviadas aos usuários)	58
include/Sistema.hpp (Define a classe Sistema, o orquestrador central de toda a aplicação de caronas)	60
include/Solicitacao.hpp (Define a classe Solicitacao, que modela a requisição de um passageiro para entrar em uma carona)	63
include/Usuario.hpp (Define a classe Usuario, a classe base polimórfica para todos os tipos de usuários no sistema)	66
include/Veiculo.hpp (Define a classe Veiculo, que encapsula os dados de um veículo cadastrado no sistema)	68
include/Zona.hpp (Define os enumeradores geográficos utilizados para classificar as rotas das caronas)	70

Documentação da classe

Referência à classe `ufmg_carona::AppExcecao`

Classe base para todas as exceções específicas da aplicação.

```
#include <Excecoes.hpp>
```

Diagrama de heranças da classe `ufmg_carona::AppExcecao`

Membros públicos

- `AppExcecao` (const std::string &message)
-

Descrição detalhada

Classe base para todas as exceções específicas da aplicação.

Nota

Herdar de `std::runtime_error` permite que essas exceções se integrem ao sistema de exceções padrão do C++. Todas as exceções do projeto devem herdar desta classe para permitir a captura genérica de erros da aplicação através de um bloco `catch (const AppExcecao& e)`.

A documentação para esta classe foi gerada a partir do seguinte ficheiro:

- `include/Excecoes.hpp`

Referência à classe

ufmg_carona::AutenticacaoFalhouException

Exceção lançada quando uma tentativa de login falha.

```
#include <Excecoes.hpp>
```

Diagrama de heranças da classe ufmɡ_carona::AutenticacaoFalhouException

Outros membros herdados

Membros públicos herdados de ufmɡ_carona::AppExcecao

- **AppExcecao** (const std::string &message)
-

Descrição detalhada

Exceção lançada quando uma tentativa de login falha.

Nota

Deve ser utilizada quando as credenciais (CPF e senha) fornecidas pelo usuário não correspondem a um registro válido no sistema.

A documentação para esta classe foi gerada a partir do seguinte ficheiro:

- include/**Excecoes.hpp**

Referência à classe `ufmg_carona::Avaliacao`

Modela uma avaliação que um usuário faz a outro após uma carona. [cite: 18].

```
#include <Avaliacao.hpp>
```

Membros públicos

- **Avaliacao** (int nota, std::string comentario, **Usuario** *avaliador, **Usuario** *avaliado, **Carona** *carona_ref)
Construtor da classe Avaliacao.
- int **get_nota** () const
Obtém a nota da avaliação.
- const std::string & **get_comentario** () const
Obtém o comentário da avaliação.
- **Usuario** * **get_avaliador** () const
Obtém o usuário que realizou a avaliação.
- **Usuario** * **get_avaliado** () const
Obtém o usuário que foi avaliado.
- **Carona** * **get_carona_referencia** () const
Obtém a carona a qual a avaliação se refere.

Atributos Privados

- int **_nota**
Nota da avaliação. Deve ser um valor inteiro em uma escala pré-definida (ex: 1 a 5).
- std::string **_comentario**
Comentário textual opcional que detalha a avaliação.
- **Usuario** * **_avaliador**
Ponteiro para o usuário que realizou a avaliação.
- **Usuario** * **_avaliado**
Ponteiro para o usuário que foi o alvo da avaliação.
- **Carona** * **_carona_referencia**
Ponteiro para a carona à qual esta avaliação se refere.

Descrição detalhada

Modela uma avaliação que um usuário faz a outro após uma carona. [cite: 18].

[cite_start]

Nota

Esta classe armazena ponteiros brutos para os objetos **Usuario** e **Carona**. Ela não gerencia o tempo de vida (ownership) desses objetos, ou seja, a [cite_start]responsabilidade pela alocação e liberação da memória é externa. [cite: 30]

Documentação dos Construtores & Destrutor

ufmg_carona::Avaliacao::Avaliacao (int nota, std::string comentario, Usuario * avaliador, Usuario * avaliado, Carona * carona_ref)

Construtor da classe **Avaliacao**.

Parâmetros

<i>nota</i>	A nota numérica da avaliação.
<i>comentario</i>	O comentário textual da avaliação.
<i>avaliador</i>	Ponteiro para o usuário que está fazendo a avaliação.
<i>avaliado</i>	Ponteiro para o usuário que está sendo avaliado.
<i>carona_ref</i>	Ponteiro para a carona de referência.

Documentação das funções

Usuario * ufmg_carona::Avaliacao::get_avaliado () const

Obtém o usuário que foi avaliado.

Retorna

Um ponteiro para o objeto **Usuario** do avaliado.

Usuario * ufmg_carona::Avaliacao::get_avaliador () const

Obtém o usuário que realizou a avaliação.

Retorna

Um ponteiro para o objeto **Usuario** do avaliador.

Carona * ufmg_carona::Avaliacao::get_carona_referencia () const

Obtém a carona a qual a avaliação se refere.

Retorna

Um ponteiro para o objeto **Carona** de referência.

const std::string & ufmg_carona::Avaliacao::get_comentario () const

Obtém o comentário da avaliação.

Retorna

Uma referência constante para a string do comentário, evitando cópias desnecessárias.

int ufmg_carona::Avaliacao::get_nota () const

Obtém a nota da avaliação.

Retorna

A nota como um valor inteiro.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- include/Avaliacao.hpp
- src/Avaliacao.cpp

Referência à classe `ufmg_carona::Carona`

A entidade central do sistema, agregando todas as informações de uma carona.

```
#include <Carona.hpp>
```

Membros públicos

- **Carona** (std::string origem_nome, std::string destino_nome, **Zona** origem_zona, **Zona** destino_zona, **UFMGPosicao** ufmg_posicao, std::string data, **Usuario** *motorista, **Veiculo** *veiculo_usado, bool apenas_mulheres, **TipoCarona** tipo)
*Construtor da classe **Carona**.*

- `int get_id () const`
Obtém o ID único desta carona.
- `Usuario * get_motorista () const`
Obtém o motorista desta carona.
- `Veiculo * get_veiculo_usado () const`
Obtém o veículo utilizado nesta carona.
- `const std::string & get_origem () const`
Obtém o nome do local de origem.
- `const std::string & get_destino () const`
Obtém o nome do local de destino.
- `Zona get_origem_zona () const`
Obtém a zona de origem da carona.
- `Zona get_destino_zona () const`
Obtém a zona de destino da carona.
- `UFMGPosicao get_ufmg_posicao () const`
Obtém o ponto de referência na UFMG.
- `StatusCarona get_status () const`
Obtém o status atual da carona.
- `void set_status (StatusCarona novo_status)`
Atualiza o status da carona.
- `const std::string & get_data_hora () const`
Obtém a data e hora de partida.
- `int get_vagas_disponiveis () const`
Obtém o número de vagas ainda disponíveis.

- **bool get_apenas_mulheres () const**
Verifica se a carona é exclusiva para mulheres.
- **void exibir_info () const**
Exibe no console um resumo das informações da carona.
- **void exibir_info_detalhada () const**
Exibe no console informações detalhadas da carona, incluindo passageiros.
- **void adicionar_solicitacao (Solicitacao *solicitacao)**
Adiciona uma nova solicitação à lista de pendências da carona.
- **const std::vector< Solicitacao * > & get_solicitacoes_pendentes () const**
Obtém a lista de solicitações pendentes.
- **bool tem_solicitacoes_pendentes () const**
Verifica se existem solicitações pendentes para esta carona.
- **void adicionar_passageiro (Usuario *passageiro)**
Adiciona um passageiro à lista de confirmados e decrementa as vagas.
- **void remover_passageiro (Usuario *passageiro)**
Remove um passageiro da lista de confirmados e incrementa as vagas.

Membros públicos estáticos

- **static int gerar_proximo_id ()**
Gera e retorna o próximo ID único para uma nova carona.

Atributos Privados

- **int _id**
*Identificador numérico único para cada instância de **Carona**.*
- **std::string _origem_nome**
Descrição textual do local de partida (ex: "Shopping Del Rey").
- **std::string _destino_nome**
Descrição textual do local de destino (ex: "Reitoria UFMG").
- **Zona _origem_zona**
Zona geográfica da cidade onde a carona se inicia.
- **Zona _destino_zona**
Zona geográfica da cidade onde a carona termina.
- **UFMGPosicao _ufmg_posicao**
Ponto de encontro ou dispersão dentro do campus da UFMG.

- **std::string _data_hora_partida**
Data e hora de partida da carona, no formato "DD/MM/AAAA HH:MM".
- **Usuario * _motorista**
*Ponteiro para o objeto **Usuario** que está oferecendo a carona.*
- **Veiculo * _veiculo_usado**
*Ponteiro para o objeto **Veiculo** utilizado na carona.*
- **std::vector< Usuario * > _passageiros**
*Vetor de ponteiros para os objetos **Usuario** dos passageiros confirmados.*
- **std::vector< Solicitacao * > _solicitacoes_pendentes**
Vetor de ponteiros para as solicitações de participação pendentes de aprovação.
- **int _vagas_disponiveis**
Número de assentos ainda disponíveis na carona.
- **bool _apenas_mulheres**
Flag que indica se a carona é exclusiva para mulheres.
- **StatusCarona _status**
O estado atual da carona no seu ciclo de vida.
- **TipoCarona _tipo**
O tipo da carona (agendada ou imediata).

Atributos Privados Estáticos

- **static int _proximo_id = 1**
Contador estático para garantir a unicidade dos IDs gerados para novas caronas.

Descrição detalhada

A entidade central do sistema, agregando todas as informações de uma carona.

Nota

Esta classe armazena ponteiros brutos (raw pointers) para várias outras entidades (**Usuario**, **Veiculo**, **Solicitacao**). Ela não gerencia o tempo de vida (ownership) desses objetos. A responsabilidade pela alocação e liberação da memória é externa a esta classe, tipicamente controlada por uma classe "Gerenciador".

Documentação dos Construtores & Destrutor

ufmg_carona::Carona::Carona (std::string origem_nome, std::string destino_nome, Zona origem_zona, Zona destino_zona, UFMGPosicao ufmg_posicao, std::string data, Usuario * motorista, Veiculo * veiculo_usado, bool apenas_mulheres, TipoCarona tipo)

Construtor da classe **Carona**.

Parâmetros

<i>origem_nome</i>	Nome do local de origem.
<i>destino_nome</i>	Nome do local de destino.
<i>origem_zona</i>	Zona da cidade de origem.
<i>destino_zona</i>	Zona da cidade de destino.
<i>ufmg_posicao</i>	Ponto de referência na UFMG.
<i>data</i>	Data e hora de partida no formato "DD/MM/AAAA HH:MM".
<i>motorista</i>	Ponteiro para o usuário motorista.
<i>veiculo_usado</i>	Ponteiro para o veículo a ser utilizado.
<i>apenas_mulheres</i>	True se a carona for exclusiva para mulheres, false caso contrário.
<i>tipo</i>	O tipo da carona (AGENDADA ou IMEDIATA).

Documentação das funções

void ufmg_carona::Carona::adicionar_passageiro (Usuario * passageiro)

Adiciona um passageiro à lista de confirmados e decrementa as vagas.

Parâmetros

<i>passageiro</i>	Ponteiro para o objeto Usuario a ser adicionado.
-------------------	---

void ufmg_carona::Carona::adicionar_solicitacao (Solicitacao * solicitacao)

Adiciona uma nova solicitação à lista de pendências da carona.

Parâmetros

<i>solicitacao</i>	Ponteiro para o objeto Solicitacao a ser adicionado.
--------------------	---

int ufmg_carona::Carona::gerar_proximo_id () [static]

Gera e retorna o próximo ID único para uma nova carona.

Retorna

Um inteiro representando o próximo ID.

bool ufmg_carona::Carona::get_apenas_mulheres () const

Verifica se a carona é exclusiva para mulheres.

Retorna

True se for apenas para mulheres, false caso contrário.

const std::string & ufmg_carona::Carona::get_data_hora () const

Obtém a data e hora de partida.

Retorna

Referência constante para a string com a data e hora.

const std::string & ufmg_carona::Carona::get_destino () const

Obtém o nome do local de destino.

Retorna

Referência constante para a string de destino.

Zona ufmg_carona::Carona::get_destino_zona () const

Obtém a zona de destino da carona.

Retorna

O enum **Zona** correspondente ao destino.

int ufmg_carona::Carona::get_id () const

Obtém o ID único desta carona.

Retorna

O ID da carona.

Usuario * ufmg_carona::Carona::get_motorista () const

Obtém o motorista desta carona.

Retorna

Ponteiro para o objeto **Usuario** do motorista.

const std::string & ufmg_carona::Carona::get_origem () const

Obtém o nome do local de origem.

Retorna

Referência constante para a string de origem.

Zona ufmg_carona::Carona::get_origem_zona () const

Obtém a zona de origem da carona.

Retorna

O enum **Zona** correspondente à origem.

const std::vector< Solicitacao * > & ufmg_carona::Carona::get_solicitacoes_pendentes () const

Obtém a lista de solicitações pendentes.

Retorna

Referência constante para o vetor de ponteiros de **Solicitacao**.

StatusCarona ufmg_carona::Carona::get_status () const

Obtém o status atual da carona.

Retorna

O enum **StatusCarona** correspondente.

UFMGPosicao ufmg_carona::Carona::get_ufmg_posicao () const

Obtém o ponto de referência na UFMG.

Retorna

O enum **UFMGPosicao** correspondente.

int ufmg_carona::Carona::get_vagas_disponiveis () const

Obtém o número de vagas ainda disponíveis.

Retorna

O número de vagas.

Veiculo * ufmg_carona::Carona::get_veiculo_usado () const

Obtém o veículo utilizado nesta carona.

Retorna

Ponteiro para o objeto **Veiculo**.

void ufmg_carona::Carona::remover_passageiro (Usuario * passageiro)

Remove um passageiro da lista de confirmados e incrementa as vagas.

Parâmetros

<i>passageiro</i>	Ponteiro para o objeto Usuario a ser removido.
-------------------	---

void ufmg_carona::Carona::set_status (StatusCarona novo_status)

Atualiza o status da carona.

Parâmetros

<i>novo_status</i>	O novo estado para a carona.
--------------------	------------------------------

bool ufmg_carona::Carona::tem_solicitacoes_pendentes () const

Verifica se existem solicitações pendentes para esta carona.

Retorna

True se houver solicitações, false caso contrário.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- include/Carona.hpp
- src/Carona.cpp

Referência à classe `ufmg_carona::CaronaFactory`

Implementa o padrão de projeto "Factory Method" para a criação de objetos **Carona**.

```
#include <CaronaFactory.hpp>
```

Membros públicos estáticos

- `static Carona criar_carona (std::string origem_nome, std::string destino_nome, Zona origem_zona, Zona destino_zona, UFMGPosicao ufmg_posicao, std::string data, Usuario *motorista, Veiculo *veiculo_usado, bool apenas_mulheres, TipoCarona tipo)`
*Cria e retorna uma nova instância de um objeto **Carona**.*

Descrição detalhada

Implementa o padrão de projeto "Factory Method" para a criação de objetos **Carona**.

- Esta classe abstrai e centraliza o processo de instanciação de caronas, permitindo que a lógica de criação seja isolada e facilmente mantida ou estendida no futuro, sem que o código cliente precise conhecer os detalhes da construção de uma **Carona**.
-
- **Nota**
Como todos os seus métodos são estáticos, esta classe não foi projetada para ser instanciada. Ela serve como um utilitário de construção.

Documentação das funções

Carona `ufmg_carona::CaronaFactory::criar_carona (std::string origem_nome, std::string destino_nome, Zona origem_zona, Zona destino_zona, UFMGPosicao ufmg_posicao, std::string data, Usuario * motorista, Veiculo * veiculo_usado, bool apenas_mulheres, TipoCarona tipo) [static]`

Cria e retorna uma nova instância de um objeto **Carona**.

◦

◦ **Parâmetros**

<i>origem_nome</i>	Nome do local de origem.
<i>destino_nome</i>	Nome do local de destino.
<i>origem_zona</i>	Zona da cidade de origem.
<i>destino_zona</i>	Zona da cidade de destino.
<i>ufmg_posicao</i>	Ponto de referência na UFMG.
<i>data</i>	Data e hora de partida no formato "DD/MM/AAAA HH:MM".
<i>motorista</i>	Ponteiro para o usuário motorista.
<i>veiculo_usado</i>	Ponteiro para o veículo a ser utilizado.
<i>apenas_mulheres</i>	True se a carona for exclusiva para mulheres, false caso contrário.
<i>tipo</i>	O tipo da carona (AGENDADA ou IMEDIATA).

◦ **Retorna**

Um objeto **Carona** totalmente inicializado.

◦

- **Nota**

Os ponteiros para motorista e veiculo_usado são repassados para o construtor da **Carona**.
A fábrica não assume a propriedade (ownership) desses ponteiros.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- include/CaronaFactory.hpp
- src/CaronaFactory.cpp

Referência à classe `ufmg_carona::CaronaLotadaException`

Exceção lançada ao tentar solicitar participação em uma carona sem vagas.

```
#include <Excecoes.hpp>
```

Diagrama de heranças da classe `ufmg_carona::CaronaLotadaException`

Outros membros herdados

Membros públicos herdados de `ufmg_carona::AppExcecao`

- `AppExcecao` (const std::string &message)

Descrição detalhada

Exceção lançada ao tentar solicitar participação em uma carona sem vagas.

A documentação para esta classe foi gerada a partir do seguinte ficheiro:

- `include/Excecoes.hpp`

Referência à classe

ufmg_carona::ComandoInvalidoException

Exceção lançada quando o sistema recebe um comando de usuário não reconhecido.

```
#include <Excecoes.hpp>
```

Diagrama de heranças da classe ufmɡ_carona::ComandoInvalidoException

Membros públicos

- **ComandoInvalidoException** (const std::string &comando)
Construtor da exceção de comando inválido.

Membros públicos herdados de ufmɡ_carona::AppExcecao

- **AppExcecao** (const std::string &message)

Descrição detalhada

Exceção lançada quando o sistema recebe um comando de usuário não reconhecido.

Nota

Útil em interfaces de linha de comando (CLI) para sinalizar que a entrada do usuário não corresponde a nenhuma ação válida.

Documentação dos Construtores & Destrutor

ufmg_carona::ComandoInvalidoException::ComandoInvalidoException (const std::string & comando) `[inline]`

Construtor da exceção de comando inválido.

Parâmetros

<i>comando</i>	A string exata do comando que foi inserido pelo usuário e não foi reconhecido.
----------------	--

A documentação para esta classe foi gerada a partir do seguinte ficheiro:

- include/Excecoes.hpp

Referência à classe ufmg_carona::Motorista

Representa um usuário que também pode atuar como motorista, oferecendo caronas.

```
#include <Motorista.hpp>
```

Diagrama de heranças da classe ufmg_carona::Motorista

Membros públicos

- **Motorista** (std::string nome, std::string cpf, std::string telefone, std::string data_nascimento, std::string email, std::string senha, **Genero** genero, std::string vinculo_tipo, std::string detalhe_vinculo, std::string cnh_numero)
*Construtor da classe **Motorista**.*
- **~Motorista** () override
*Destrutor da classe **Motorista**.*
- void **imprimir_perfil** () const override
Sobrescreve o método da classe base para exibir o perfil completo do motorista.
- void **adicionar_veiculo** (**Veiculo** *veiculo)
Adiciona um veículo à lista do motorista.
- bool **is_motorista** () const override
Identifica o tipo do objeto polimorficamente.
- const std::string & **get_cnh_numero** () const
Obtém o número da CNH do motorista.
- const std::vector< **Veiculo** * > & **get_veiculos** () const
Obtém a lista de veículos do motorista.
- **Veiculo** * **buscar_veiculo_por_placa** (const std::string &placa) const
Busca um veículo na lista do motorista pela placa.
- **Veiculo** * **buscar_veiculo_por_indice** (size_t indice) const
Busca um veículo na lista do motorista pelo seu índice no vetor.
- bool **remover_veiculo** (const std::string &placa)
Remove um veículo da lista do motorista com base na placa.

Membros públicos herdados de ufmg_carona::Usuario

- **Usuario** (std::string nome, std::string cpf, std::string telefone, std::string data_nascimento, std::string email, std::string senha, **Genero** genero, std::string vinculo_tipo, std::string detalhe_vinculo)
*Construtor da classe base **Usuario**.*
- virtual **~Usuario** ()

Destrutor virtual.

- `std::string get_vinculo () const`
- `std::string get_vinculo_raw () const`
- `std::string get_detalhe_vinculo () const`
- `const std::string & get_email () const`
- `const std::string & get_senha () const`
- `Genero get_genero () const`
- `const std::string & get_cpf () const`
- `const std::string & get_nome () const`
- `const std::string & get_telefone () const`
- `const std::string & get_data_nascimento () const`
- `bool verificar_senha (const std::string &senha) const`
Compara a senha fornecida com a senha armazenada.
- `double get_media_avaliacoes () const`
Calcula e retorna a média das notas das avaliações recebidas.
- `std::string get_medalha () const`
Determina a "medalha" de reputação do usuário com base na média de suas avaliações.
- `void adicionar_avaliacao_recebida (Avaliacao *avaliacao)`
Adiciona uma nova avaliação recebida à lista do usuário.
- `void adicionar_notificacao (const Notificacao ¬ificacao)`
Adiciona uma nova notificação à lista do usuário.
- `const std::vector< Notificacao > & get_notificacoes () const`
Obtém acesso de leitura à lista de notificações.
- `std::vector< Notificacao > & get_notificacoes_mutavel ()`
Obtém acesso de escrita à lista de notificações.
- `void set_email (const std::string &email)`
- `void set_telefone (const std::string &telefone)`
- `void set_senha (const std::string &senha)`
- `void set_genero (Genero genero)`

Atributos Privados

- `std::vector< Veiculo * > _veiculos`
Vetor de ponteiros para os veículos pertencentes a este motorista.
- `std::string _cnh_numero`
Número de registro da Carteira Nacional de Habilitação (CNH) do motorista.

Outros membros herdados

Atributos Protegidos herdados de `ufmg_carona::Usuario`

- `std::string _nome`
- `std::string _cpf`

- `std::string _email`
- `std::string _senha`
- `std::string _telefone`
- `std::string _data_nascimento`
Data de nascimento do usuário, armazenada no formato "DD/MM/AAAA".
- **Genero _genero**
- `std::vector< Avaliacao * > _avaliacoes_recebidas`
Vetor de ponteiros não-proprietários para as avaliações recebidas pelo usuário.
- `std::vector< Notificacao > _notificacoes`
Vetor de notificações do usuário. Armazenadas por valor.
- `std::string _vinculo_tipo`
Tipo de vínculo com a UFMG (ex: "Aluno", "Professor", "Funcionario").
- `std::string _detalhe_vinculo`
Detalhe do vínculo (ex: "Ciência da Computação", "Departamento de Química").

Descrição detalhada

Representa um usuário que também pode atuar como motorista, oferecendo caronas.

- Esta classe herda de **Usuario**, adicionando funcionalidades e atributos específicos de um motorista, como o número da CNH e uma lista de veículos cadastrados.
-
- **Nota**
A classe **Motorista** armazena uma coleção de ponteiros brutos para objetos **Veiculo**. Ela **não gerencia o tempo de vida (ownership)** desses objetos. A responsabilidade pela criação e destruição dos veículos é de um componente externo (ex: GerenciadorVeiculos).

Documentação dos Construtores & Destrutor

ufmg_carona::Motorista::Motorista (std::string nome, std::string cpf, std::string telefone, std::string data_nascimento, std::string email, std::string senha, Genero genero, std::string vinculo_tipo, std::string detalhe_vinculo, std::string cnh_numero)

Construtor da classe **Motorista**.

◦

◦ **Parâmetros**

<i>nome</i>	Nome completo do usuário.
<i>cpf</i>	Cadastro de Pessoa Física (deve ser único).
<i>telefone</i>	Número de telefone para contato.
<i>data_nascimento</i>	Data de nascimento no formato "DD/MM/AAAA".
<i>email</i>	Endereço de e-mail (deve ser único).
<i>senha</i>	Senha para acesso ao sistema.
<i>genero</i>	Identidade de gênero do usuário.
<i>vinculo_tipo</i>	Tipo de vínculo com a UFMG (ex: "Aluno", "Professor").

<i>detalhe_vinculo</i>	Detalhe do vínculo (ex: "Ciência da Computação", "DCC").
<i>cnh_numero</i>	Número da CNH do motorista.

ufmg_carona::Motorista::~~Motorista () [override]

Destrutor da classe **Motorista**.

Nota

Este destrutor **não** libera a memória dos objetos **Veiculo** apontados pelos ponteiros no vetor `_veiculos`. Ele apenas destrói o vetor de ponteiros.

Documentação das funções

void ufmg_carona::Motorista::adicionar_veiculo (Veiculo * veiculo)

Adiciona um veículo à lista do motorista.

Parâmetros

<i>veiculo</i>	Ponteiro para o objeto Veiculo a ser adicionado. A posse não é transferida.
----------------	--

Veiculo * ufmg_carona::Motorista::buscar_veiculo_por_indice (size_t indice) const

Busca um veículo na lista do motorista pelo seu índice no vetor.

Parâmetros

<i>indice</i>	O índice do veículo a ser buscado.
---------------	------------------------------------

Retorna

Um ponteiro para o objeto **Veiculo** se o índice for válido, caso contrário, `nullptr`.

Nota

O chamador deve garantir que o índice está dentro dos limites do vetor.

Veiculo * ufmg_carona::Motorista::buscar_veiculo_por_placa (const std::string & placa) const

Busca um veículo na lista do motorista pela placa.

Parâmetros

<i>placa</i>	A placa do veículo a ser buscado.
--------------	-----------------------------------

Retorna

Um ponteiro para o objeto **Veiculo** se encontrado, caso contrário, `nullptr`.

const std::string & ufmg_carona::Motorista::get_cnh_numero () const

Obtém o número da CNH do motorista.

Retorna

Referência constante para a string do número da CNH.

const std::vector< Veiculo * > & ufmg_carona::Motorista::get_veiculos () const

Obtém a lista de veículos do motorista.

Retorna

Referência constante para o vetor de ponteiros de **Veiculo**.

void ufmg_carona::Motorista::imprimir_perfil () const [override], [virtual]

Sobrescreve o método da classe base para exibir o perfil completo do motorista.

Nota

Inclui informações específicas do motorista, como a CNH e a lista de veículos.

Reimplementado de **ufmg_carona::Usuario** (p. 42).

bool ufmg_carona::Motorista::is_motorista () const [override], [virtual]

Identifica o tipo do objeto polimorficamente.

Retorna

Sempre `true` para instâncias da classe **Motorista**.

Reimplementado de **ufmg_carona::Usuario** (p. 42).

bool ufmg_carona::Motorista::remover_veiculo (const std::string & placa)

Remove um veículo da lista do motorista com base na placa.

Parâmetros

<i>placa</i>	A placa do veículo a ser removido.
--------------	------------------------------------

Retorna

`true` se o veículo foi encontrado e removido, `false` caso contrário.

Nota

Esta função apenas remove o ponteiro do vetor, não deleta o objeto **Veiculo**.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- `include/Motorista.hpp`
- `src/Motorista.cpp`

Referência à classe `ufmg_carona::Notificacao`

Encapsula uma mensagem e seu estado (lida/não lida) para comunicação com o usuário. Esta classe é usada para informar os usuários sobre eventos importantes, como a aceitação de uma solicitação de carona ou uma nova avaliação recebida.

```
#include <Notificacao.hpp>
```

Membros públicos

- **Notificacao** (std::string mensagem)
*Constrói um novo objeto **Notificacao**.*
- void **marcar_como_lida** ()
Altera o estado da notificação para "lida".
- const std::string & **get_mensagem** () const
Obtém a mensagem da notificação.
- bool **is_lida** () const
Verifica se a notificação foi marcada como lida.

Atributos Privados

- std::string **_mensagem**
O conteúdo textual da notificação a ser exibido ao usuário.
- bool **_lida**
Flag booleana que indica se a notificação já foi visualizada pelo usuário.

Descrição detalhada

Encapsula uma mensagem e seu estado (lida/não lida) para comunicação com o usuário. Esta classe é usada para informar os usuários sobre eventos importantes, como a aceitação de uma solicitação de carona ou uma nova avaliação recebida.

Documentação dos Construtores & Destrutor

`ufmg_carona::Notificacao::Notificacao (std::string mensagem)`

Constrói um novo objeto **Notificacao**.

Parâmetros

<code>mensagem</code>	O texto da mensagem para a notificação.
-----------------------	---

Nota

Uma notificação é sempre criada com o estado "não lida" (`_lida = false`).

Documentação das funções

const std::string & ufmg_carona::Notificacao::get_mensagem () const

Obtém a mensagem da notificação.

Retorna

Uma referência constante para a string da mensagem, evitando cópias.

bool ufmg_carona::Notificacao::is_lida () const

Verifica se a notificação foi marcada como lida.

Retorna

`true` se a notificação foi lida, `false` caso contrário.

void ufmg_carona::Notificacao::marcar_como_lida ()

Altera o estado da notificação para "lida".

Nota

Esta é uma operação idempotente; chamar este método múltiplas vezes não altera o estado após a primeira chamada.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- `include/Notificacao.hpp`
- `src/Notificacao.cpp`

Referência à classe `ufmg_carona::Sistema`

A classe principal que gerencia o estado, os dados e os fluxos de interação do usuário. Atua como um orquestrador para toda a aplicação, inicializando subsistemas, controlando o loop principal e garantindo a persistência dos dados.

```
#include <Sistema.hpp>
```

Membros públicos

- **Sistema ()**
*Construtor do **Sistema**. Inicializa os contêineres e carrega os dados iniciais dos arquivos.*
- **~Sistema ()**
*Destrutor do **Sistema**. Garante que todos os dados sejam salvos e, crucialmente, libera toda a memória alocada dinamicamente para os objetos nos vetores de ponteiros.*
- **void executar ()**
Ponto de entrada público que inicia e executa o loop principal da aplicação.

Membros privados

- Persistência de Dados **void carregar_dados_iniciais ()**
Carrega todos os dados de arquivos de texto no início da execução.
- **void salvar_dados_usuarios ()**
Salva os dados de usuários em arquivo.
- **void salvar_dados_veiculos ()**
Salva os dados de veículos em arquivo.
- **void salvar_dados_caronas ()**
Salva as caronas ativas em arquivo.
- **void carregar_dados_caronas ()**
Carrega as caronas ativas de arquivo.
- **void carregar_dados_solicitacoes ()**
Carrega as solicitações de arquivo.
- **void salvar_dados_solicitacoes ()**
Salva as solicitações em arquivo.
- **void salvar_dados_avaliacoes ()**
Salva as avaliações em arquivo.
- **void carregar_dados_avaliacoes ()**
Carrega as avaliações de arquivo.
- **Buscas Internas****Usuario * buscar_usuario_por_cpf (const std::string &cpf)**

Procura um usuário pelo seu CPF.

- **Carona * buscar_carona_por_id** (int id)
Procura uma carona pelo seu ID.
- **Veiculo * buscar_veiculo_por_placa_motorista** (Motorista *motorista, const std::string &placa)
Procura um veículo pela placa dentro da lista de um motorista específico.
- std::tuple< bool, std::string, std::string, std::string, std::string > **buscar_dados_ufmg_por_cpf** (const std::string &cpf_buscado)
Simula uma consulta a uma base de dados externa da UFMG.
- Interface com Usuário (Menus)void **exibir_menu_inicial_nao_logado** ()
Exibe o menu para usuários não autenticados.
- void **exibir_menu_logado** ()
Exibe o menu principal para usuários autenticados.
- void **exibir_menu_passageiro** ()
Exibe as opções específicas para o modo passageiro.
- void **exibir_menu_motorista** ()
Exibe as opções específicas para o modo motorista.
- void **exibir_menu** ()
Orquestra qual menu deve ser exibido com base no estado de login.
- Processamento de Comandosvoid **processar_comando_logado** (const std::string &comando_str)
Processa uma entrada de comando do usuário logado.
- Fluxos de Interação do Usuáriovoid **fluxo_cadastro** ()
- void **fluxo_login** ()
- void **fluxo_logout** ()
- void **fluxo_oferecer_carona** ()
- void **fluxo_solicitar_carona** ()
- void **fluxo_status_caronas** ()
- void **fluxo_cadastrar_veiculo** ()
- void **fluxo_editar_perfil** ()
- void **fluxo_passageiro_menu** ()
- void **fluxo_motorista_menu** ()
- void **fluxo_editar_perfil_ou_veiculos** ()
- void **fluxo_tornar_motorista** ()
- void **fluxo_gerenciar_veiculos** ()
- void **fluxo_editar_veiculo** (Motorista *motorista)
- void **fluxo_excluir_veiculo** (Motorista *motorista)
- void **fluxo_gerenciar_caronas** ()
- void **fluxo_solicitacoes_pendentes_motorista** ()
- void **fluxo_minhas_caronas** (Motorista *motorista_logado)
- void **fluxo_avaliacoes** ()
- void **fluxo_avaliar_carona_passageiro** ()
- void **fluxo_avaliar_passageiros_motorista** ()

- void **exibir_minhas_avaliacoes_recebidas** ()
- void **exibir_avaliacoes_que_fiz** ()
- Lógica de Negócio e Helpers void **enviar_notificacao** (**Usuario** *usuario, const std::string &mensagem, bool enviar_para_motorista=true)
Envia uma notificação para um usuário.
- bool **pode_solicitar_carona** (**Usuario** *passageiro, const **Carona** &carona)
Verifica se um passageiro cumpre os pré-requisitos para solicitar uma carona.
- void **cancelar_outras_solicitacoes_passageiro** (**Usuario** *passageiro, const **Carona** &carona_aceita)
Cancela automaticamente outras solicitações de um passageiro quando uma é aceita.
- void **finalizar_carona_completa** (**Carona** *carona_para_finalizar)
Finaliza uma carona e todas as suas solicitações associadas.
- void **cancelar_carona_completa** (**Carona** *carona_para_cancelar)
Cancela uma carona e notifica todos os envolvidos.
- void **remover_caronas_passadas** ()
Remove da memória as caronas que já aconteceram.
- Utilitários de Input/Output e Conversão int **coletar_int_input** (const std::string &prompt, int min_val, int max_val)
- std::string **coletar_string_input** (const std::string &prompt)
- **Zona** **coletar_zona_input** (const std::string &prompt)
- **UFMGPosicao** **coletar_ufmg_posicao_input** (const std::string &prompt)
- std::string **zona_to_string** (**Zona** z) const
- **Zona** **string_to_zona** (const std::string &s) const
- std::string **ufmg_posicao_to_string** (**UFMGPosicao** up) const
- **UFMGPosicao** **string_to_ufmg_posicao** (const std::string &s) const
- std::tm **parse_datetime_string** (const std::string &dt_str) const
- std::string **get_current_datetime_string** () const
- bool **is_datetime_in_past** (const std::string &dt_str) const

Atributos Privados

- Contêineres de Dados Principais std::vector< **Usuario** * > **_usuarios**
*Vetor de ponteiros para todos os usuários cadastrados. O **Sistema** é o dono desses objetos.*
- std::vector< **Carona** > **_caronas**
Vetor de todos os objetos de carona. Armazenados por valor, o gerenciamento de memória é automático.
- std::vector< **Solicitacao** * > **_solicitacoes**
*Vetor de ponteiros para todas as solicitações de carona. O **Sistema** é o dono desses objetos.*
- std::vector< **Avaliacao** * > **_avaliacoes_globais**
*Vetor de ponteiros para todas as avaliações realizadas no sistema. O **Sistema** é o dono desses objetos.*
- Estado da Aplicação **Usuario** * **_usuario_logado**

Ponteiro não-proprietário para o usuário atualmente logado na sessão. `nullptr` se ninguém estiver logado.

- Utilitários de Conversão `std::map< int, Zona > _int_para_zona`
Mapas para conversão entre enums e tipos primitivos/strings, usados para I/O e persistência.
- `std::map< Zona, std::string > _zona_para_string`
- `std::map< int, UFMGPosicao > _int_para_ufmg_posicao`
- `std::map< UFMGPosicao, std::string > _ufmg_posicao_para_string`

Descrição detalhada

A classe principal que gerencia o estado, os dados e os fluxos de interação do usuário. Atua como um orquestrador para toda a aplicação, inicializando subsistemas, controlando o loop principal e garantindo a persistência dos dados.

Nota

Esta classe é a **proprietária (owner)** de todos os objetos alocados dinamicamente cujos ponteiros são armazenados em seus vetores (ex: `_usuarios`, `_solicitacoes`, `_avaliacoes_globais`). Seu destrutor é responsável por liberar toda essa memória para evitar memory leaks.

Documentação das funções

Carona * ufmg_carona::Sistema::buscar_carona_por_id (int id) [private]

Procura uma carona pelo seu ID.

Retorna

Ponteiro para a **Carona** se encontrada, senão `nullptr`.

std::tuple< bool, std::string, std::string, std::string, std::string >
ufmg_carona::Sistema::buscar_dados_ufmg_por_cpf (const std::string &
cpf_buscado) [private]

Simula uma consulta a uma base de dados externa da UFMG.

Retorna

Uma tupla contendo (sucesso, nome, vínculo, detalhe_vínculo, email).

Usuario * ufmg_carona::Sistema::buscar_usuario_por_cpf (const std::string &
cpf) [private]

Procura um usuário pelo seu CPF.

Retorna

Ponteiro para o **Usuario** se encontrado, senão `nullptr`.

**Veiculo * ufmg_carona::Sistema::buscar_veiculo_por_placa_motorista (Motorista *
motorista, const std::string & placa) [private]**

Procura um veículo pela placa dentro da lista de um motorista específico.

Retorna

Ponteiro para o **Veiculo** se encontrado, senão `nullptr`.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- `include/Sistema.hpp`
- `src/Sistema.cpp`

Referência à classe `ufmg_carona::Solicitacao`

Modela uma solicitação de um passageiro para participar de uma carona. Esta classe encapsula todo o fluxo de negociação, incluindo propostas de locais e o estado final da solicitação, além de rastrear o status das avaliações pós-viagem.

```
#include <Solicitacao.hpp>
```

Membros públicos

- **Solicitacao** (**Usuario** *passageiro, **Carona** *carona, std::string local_embarque_p, std::string local_desembarque_p)
Construtor de uma nova solicitação.
- Métodos de Ação e Negociaçãovoid **aceitar** ()
Altera o status da solicitação para ACEITA.
- void **recusar** ()
Altera o status da solicitação para RECUSADA.
- void **propor_locais_motorista** (std::string local_embarque_m, std::string local_desembarque_m)
Registra uma contraproposta do motorista e altera o status para AGUARDANDO_RESPOSTA_PASSAGEIRO.
- void **aceitar_proposta_motorista** ()
Ação do passageiro para aceitar a contraproposta. Altera o status para ACEITA.
- void **recusar_proposta_motorista** ()
Ação do passageiro para recusar a contraproposta. Altera o status para RECUSADA_PROPOSTA_MOTORISTA.
- Settersvoid **set_status** (**StatusSolicitacao** novo_status)
Define manualmente o status da solicitação.
- void **set_carona** (**Carona** *new_carona_ptr)
Associa a solicitação a um novo objeto de carona.
- Getters**Usuario** * **get_passageiro** () const
- **Carona** * **get_carona** () const
- **StatusSolicitacao** **get_status** () const
- std::string **get_status_string** () const
Obtém uma representação textual do status atual.
- const std::string & **get_local_embarque_passageiro** () const
- const std::string & **get_local_desembarque_passageiro** () const
- const std::string & **get_local_embarque_motorista_proposto** () const
- const std::string & **get_local_desembarque_motorista_proposto** () const
- Métodos de Exibiçãovoid **exibir_info** () const
Exibe informações gerais da solicitação no console.
- void **exibir_para_motorista** () const

Exibe as informações da solicitação sob a perspectiva do motorista.

- Rastreamento de Avaliações `bool get_passageiro_avaliou_motorista () const`
- `void set_passageiro_avaliou_motorista (bool avaliou)`
- `bool get_motorista_avaliou_passageiro () const`
- `void set_motorista_avaliou_passageiro (bool avaliou)`

Atributos Privados

- **Usuario * _passageiro**
Ponteiro para o usuário passageiro que fez a solicitação.
- **Carona * _carona_alvo**
Ponteiro para a carona que é o alvo da solicitação.
- **StatusSolicitacao _status**
O estado atual da solicitação dentro do fluxo de negociação.
- `std::string _local_embarque_passageiro`
Ponto de encontro sugerido pelo passageiro.
- `std::string _local_desembarque_passageiro`
Ponto de desembarque sugerido pelo passageiro.
- `std::string _local_embarque_motorista_proposto`
Contraproposta de local de embarque feita pelo motorista.
- `std::string _local_desembarque_motorista_proposto`
Contraproposta de local de desembarque feita pelo motorista.
- `bool _passageiro_avaliou_motorista`
Flag para rastrear se o passageiro já avaliou o motorista após esta carona.
- `bool _motorista_avaliou_passageiro`
Flag para rastrear se o motorista já avaliou o passageiro após esta carona.

Descrição detalhada

Modela uma solicitação de um passageiro para participar de uma carona. Esta classe encapsula todo o fluxo de negociação, incluindo propostas de locais e o estado final da solicitação, além de rastrear o status das avaliações pós-viagem.

Nota

Armazena ponteiros brutos para **Usuario** and **Carona**, mas não gerencia seu tempo de vida (ownership). A responsabilidade pela memória desses objetos é externa.

Documentação dos Construtores & Destrutor

ufmg_carona::Solicitacao::Solicitacao (Usuario * passageiro, Carona * carona, std::string local_embarque_p, std::string local_desembarque_p)

Construtor de uma nova solicitação.

Parâmetros

<i>passageiro</i>	Ponteiro para o usuário que faz a solicitação.
<i>carona</i>	Ponteiro para a carona desejada.
<i>local_embarque_p</i>	Ponto de embarque sugerido pelo passageiro.
<i>local_desembarque_p</i>	Ponto de desembarque sugerido pelo passageiro.

Nota

Uma solicitação é criada com o status inicial PENDENTE.

Documentação das funções

std::string ufmg_carona::Solicitacao::get_status_string () const

Obtém uma representação textual do status atual.

Retorna

String com o nome do status.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- include/Solicitacao.hpp
- src/Solicitacao.cpp

Referência à classe ufmg_carona::Usuario

Classe base que encapsula os dados e comportamentos comuns a todos os usuários. Serve como a fundação para o polimorfismo no sistema, permitindo que diferentes tipos de usuários (como **Motorista**) sejam tratados de forma uniforme.

```
#include <Usuario.hpp>
```

Diagrama de heranças da classe ufmg_carona::Usuario

Membros públicos

- **Usuario** (std::string nome, std::string cpf, std::string telefone, std::string data_nascimento, std::string email, std::string senha, **Genero** genero, std::string vinculo_tipo, std::string detalhe_vinculo)
Construtor da classe base Usuario.
- virtual ~**Usuario** ()
Destrutor virtual.
- Getters de Informações Pessoais e de Vinculo std::string **get_vinculo** () const
- std::string **get_vinculo_raw** () const
- std::string **get_detalhe_vinculo** () const
- const std::string & **get_email** () const
- const std::string & **get_senha** () const
- **Genero** **get_genero** () const
- const std::string & **get_cpf** () const
- const std::string & **get_nome** () const
- const std::string & **get_telefone** () const
- const std::string & **get_data_nascimento** () const
- Métodos de Autenticação e Identificação bool **verificar_senha** (const std::string &senha) const
Compara a senha fornecida com a senha armazenada.
- virtual bool **is_motorista** () const
Verifica se o usuário é um motorista.
- Métodos de Avaliação e Reputação double **get_media_avaliacoes** () const
Calcula e retorna a média das notas das avaliações recebidas.
- std::string **get_medalha** () const
Determina a "medalha" de reputação do usuário com base na média de suas avaliações.
- void **adicionar_avaliacao_recebida** (**Avaliacao** *avaliacao)
Adiciona uma nova avaliação recebida à lista do usuário.
- Métodos de Perfil e Notificações virtual void **imprimir_perfil** () const
Imprime as informações do perfil do usuário no console. É virtual para ser sobrescrito.
- void **adicionar_notificacao** (const **Notificacao** ¬ificacao)
Adiciona uma nova notificação à lista do usuário.

- `const std::vector< Notificacao > & get_notificacoes () const`
Obtém acesso de leitura à lista de notificações.
- `std::vector< Notificacao > & get_notificacoes_mutavel ()`
Obtém acesso de escrita à lista de notificações.
- Setters para Modificação de Perfilvoid **set_email** (const std::string &email)
- void **set_telefone** (const std::string &telefone)
- void **set_senha** (const std::string &senha)
- void **set_genero** (**Genero** genero)

Atributos Protegidos

- Atributos do Usuáriostd::string **_nome**
- std::string **_cpf**
- std::string **_email**
- std::string **_senha**
- std::string **_telefone**
- std::string **_data_nascimento**
Data de nascimento do usuário, armazenada no formato "DD/MM/AAAA".
- **Genero _genero**
- std::vector< **Avaliacao** * > **_avaliacoes_recebidas**
Vetor de ponteiros não-proprietários para as avaliações recebidas pelo usuário.
- std::vector< **Notificacao** > **_notificacoes**
Vetor de notificações do usuário. Armazenadas por valor.
- std::string **_vinculo_tipo**
Tipo de vínculo com a UFMG (ex: "Aluno", "Professor", "Funcionario").
- std::string **_detalhe_vinculo**
Detalhe do vínculo (ex: "Ciência da Computação", "Departamento de Química").

Descrição detalhada

Classe base que encapsula os dados e comportamentos comuns a todos os usuários. Serve como a fundação para o polimorfismo no sistema, permitindo que diferentes tipos de usuários (como **Motorista**) sejam tratados de forma uniforme.

Nota

Esta classe armazena ponteiros brutos em `_avaliacoes_recebidas`. Ela **não** gerencia o tempo de vida (ownership) desses objetos **Avaliacao**. A responsabilidade pela alocação e liberação da memória é de um componente externo (ex: a classe **Sistema**).

Documentação dos Construtores & Destrutor

ufmg_carona::Usuario::Usuario (std::string nome, std::string cpf, std::string telefone, std::string data_nascimento, std::string email, std::string senha, Genero genero, std::string vinculo_tipo, std::string detalhe_vinculo)

Construtor da classe base **Usuario**.

Parâmetros

<i>nome</i>	Nome completo.
<i>cpf</i>	CPF (usado como identificador único).
<i>telefone</i>	Telefone de contato.
<i>data_nascimento</i>	Data de nascimento no formato "DD/MM/AAAA".
<i>email</i>	Endereço de e-mail (usado como identificador único).
<i>senha</i>	Senha para autenticação.
<i>genero</i>	Identidade de gênero.
<i>vinculo_tipo</i>	Tipo de vínculo com a universidade.
<i>detalhe_vinculo</i>	Detalhes do vínculo (curso, departamento, etc.).

ufmg_carona::Usuario::~~Usuario () [virtual]

Destrutor virtual.

Nota

Essencial para garantir que os destrutores de classes derivadas (como **Motorista**) sejam chamados corretamente durante a destruição polimórfica.

Documentação das funções

void ufmg_carona::Usuario::adicionar_avaliacao_recebida (Avaliacao * avaliacao)

Adiciona uma nova avaliação recebida à lista do usuário.

Parâmetros

<i>avaliacao</i>	Ponteiro não-proprietário para a avaliação.
------------------	---

std::string ufmg_carona::Usuario::get_medalha () const

Determina a "medalha" de reputação do usuário com base na média de suas avaliações.

Retorna

Uma string com o nome da medalha (ex: "Bronze", "Ouro").

double ufmg_carona::Usuario::get_media_avaliacoes () const

Calcula e retorna a média das notas das avaliações recebidas.

Retorna

A média, ou 0.0 se não houver avaliações.

std::vector< Notificacao > & ufmg_carona::Usuario::get_notificacoes_mutavel ()

Obtém acesso de escrita à lista de notificações.

*

Retorna

Uma referência mutável para o vetor de notificações.

Nota

Use com cuidado, pois permite a modificação direta do vetor interno. Útil para, por exemplo, marcar notificações como lidas.

void ufmg_carona::Usuario::imprimir_perfil () const [virtual]

Imprime as informações do perfil do usuário no console. É virtual para ser sobrescrito.

Reimplementado em **ufmg_carona::Motorista** (p. 28).

bool ufmg_carona::Usuario::is_motorista () const [virtual]

Verifica se o usuário é um motorista.

Retorna

Por padrão, retorna `false`. Classes derivadas como **Motorista** devem sobrescrever este método.

Reimplementado em **ufmg_carona::Motorista** (p. 28).

bool ufmg_carona::Usuario::verificar_senha (const std::string & senha) const

Compara a senha fornecida com a senha armazenada.

Retorna

`true` se as senhas coincidirem.

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- `include/Usuario.hpp`
- `src/Usuario.cpp`

Referência à classe `ufmg_carona::Veiculo`

Modela um veículo, contendo todas as suas informações relevantes. Esta classe atua como uma estrutura de dados para armazenar os detalhes de um veículo que pode ser utilizado em uma carona.
`#include <Veiculo.hpp>`

Membros públicos

- **`Veiculo`** (`const std::string &placa`, `const std::string &marca`, `const std::string &modelo`, `const std::string &cor`, `int lugares`)
*Construtor principal para criar um objeto **`Veiculo`** totalmente inicializado.*
 - **`Veiculo`** ()
Construtor padrão.
 - `void exibir_info () const`
Exibe as informações formatadas do veículo no console. Útil para fins de depuração e interface com o usuário.
 - `Getters`
 - `int get_lugares () const`
Obtém o número total de lugares do veículo.
 - `const std::string & get_placa () const`
 - `const std::string & get_marca () const`
 - `const std::string & get_modelo () const`
 - `const std::string & get_cor () const`
 - `Setters`
- Nota**
Note que a placa do veículo é imutável após a construção para garantir a integridade do identificador.
- `void set_marca (const std::string &marca)`
 - `void set_modelo (const std::string &modelo)`
 - `void set_cor (const std::string &cor)`
 - `void set_lugares (int lugares)`

Atributos Privados

- `std::string _placa`
Placa do veículo, usada como identificador único (ex: "ABC-1234" ou "ABCID23").
- `std::string _marca`
- `std::string _modelo`
- `std::string _cor`
- `int _total_de_lugares`
Número total de lugares no veículo, incluindo o assento do motorista.

Descrição detalhada

Modela um veículo, contendo todas as suas informações relevantes. Esta classe atua como uma estrutura de dados para armazenar os detalhes de um veículo que pode ser utilizado em uma carona.

Documentação dos Construtores & Destrutor

ufmg_carona::Veiculo::Veiculo (const std::string & placa, const std::string & marca, const std::string & modelo, const std::string & cor, int lugares)

Construtor principal para criar um objeto **Veiculo** totalmente inicializado.

Parâmetros

<i>placa</i>	A placa do veículo (identificador único).
<i>marca</i>	A fabricante do veículo (ex: "Fiat", "Chevrolet").
<i>modelo</i>	O modelo do veículo (ex: "Uno", "Onix").
<i>cor</i>	A cor predominante do veículo.
<i>lugares</i>	O número total de assentos, incluindo o do motorista.

ufmg_carona::Veiculo::Veiculo ()

Construtor padrão.

Nota

Cria um objeto **Veiculo** em um estado padrão (vazio). Útil para cenários onde um objeto precisa ser criado e seus dados preenchidos posteriormente.

Documentação das funções

int ufmg_carona::Veiculo::get_lugares () const

Obtém o número total de lugares do veículo.

Retorna

O número total de assentos (incluindo o do motorista).

A documentação para esta classe foi gerada a partir dos seguintes ficheiros:

- include/Veiculo.hpp
- src/Veiculo.cpp

Documentação do ficheiro

Referência ao ficheiro `include/Avaliacao.hpp`

Define a classe `Avaliacao`, que encapsula os dados de uma avaliação feita por um usuário a outro no contexto de uma carona específica.

```
#include <string>
```

Componentes

class `ufmg_carona::Avaliacao` *Modela uma avaliação que um usuário faz a outro após uma carona. [cite: 18].*

Descrição detalhada

Define a classe `Avaliacao`, que encapsula os dados de uma avaliação feita por um usuário a outro no contexto de uma carona específica.

Avaliacao.hpp

Ir para a documentação deste ficheiro.

```
1
2
3
4
5
6 #ifndef AVALIACAO_HPP
7 #define AVALIACAO_HPP
8
9 #include <string>
10
11 namespace ufmg carona {
12
13     // Forward declarations para evitar inclusões circulares e otimizar a compilação.
14     class Usuario;
15     class Carona;
16
17
18
19
20
21
22     class Avaliacao {
23     private:
24         int _nota;
25
26
27
28         std::string _comentario;
29
30
31
32         Usuario* _avaliador;
33
34         Usuario* _avaliado;
35
36         Carona* _carona_referencia;
37
38     public:
39         Avaliacao(int nota, std::string comentario, Usuario* avaliador, Usuario*
avaliado, Carona* carona_ref);
40
41         int get_nota() const;
42
43         const std::string& get_comentario() const;
44
45         Usuario* get_avaliador() const;
46
47         Usuario* get_avaliado() const;
48
49         Carona* get_carona_referencia() const;
50     };
51 }
52 #endif
```

Referência ao ficheiro include/Carona.hpp

Define a classe Carona, a entidade central do sistema que representa uma viagem oferecida.

```
#include <string>
#include <vector>
#include "Zona.hpp"
```

Componentes

`class ufmg_carona::Carona` A entidade central do sistema, agregando todas as informações de uma carona.

Enumerações

- enum class `ufmg_carona::TipoCarona` { AGENDADA, IMEDIATA }
Define o tipo de uma carona, se é agendada para o futuro ou imediata.
- enum class `ufmg_carona::StatusCarona` { AGUARDANDO, LOTADA, EM_VIAGEM, FINALIZADA, CANCELADA }
Representa os diferentes estados possíveis de uma carona durante seu ciclo de vida.

Descrição detalhada

Define a classe Carona, a entidade central do sistema que representa uma viagem oferecida.

Documentação dos valores da enumeração

`enum class ufmg_carona::StatusCarona [strong]`

Representa os diferentes estados possíveis de uma carona durante seu ciclo de vida.

Valores de enumerações:

AGUARDANDO	A carona está aguardando passageiros.
LOTADA	Todas as vagas foram preenchidas.
EM_VIAGEM	A carona está em andamento.
FINALIZADA	A carona foi concluída com sucesso.
CANCELADA	A carona foi cancelada pelo motorista.

`enum class ufmg_carona::TipoCarona [strong]`

Define o tipo de uma carona, se é agendada para o futuro ou imediata.

Valores de enumerações:

AGENDADA	A carona tem uma data e hora de partida futuras.
IMEDIATA	A carona está pronta para partir assim que lotar.

Carona.hpp

Ir para a documentação deste ficheiro.

```
1
2
3
4
5 #ifndef CARONA_HPP
6 #define CARONA_HPP
7
8 #include <string>
9 #include <vector>
10 #include "Zona.hpp" // Assumindo que UFMGPosicao também está aqui.
11
12 namespace ufmg_carona {
13
14     // Forward declarations para evitar dependências circulares e otimizar a compilação.
15     class Usuario;
16     class Solicitacao;
17     class Veiculo;
18
19
20
21     enum class TipoCarona {
22         AGENDADA,
23         IMEDIATA
24     };
25
26     enum class StatusCarona {
27         AGUARDANDO,
28         LOTADA,
29         EM_VIAGEM,
30         FINALIZADA,
31         CANCELADA
32     };
33
34     class Carona {
35     private:
36         int _id;
37
38         static int _proximo_id;
39
40         std::string _origem_nome;
41
42         std::string _destino_nome;
43
44         Zona _origem_zona;
45
46         Zona _destino_zona;
47
48         UFMGPosicao _ufmg_posicao;
49
50         std::string _data_hora_partida;
51
52         Usuario* _motorista;
53
54         Veiculo* _veiculo_usado;
55
56         std::vector<Usuario*> _passageiros;
57
58         std::vector<Solicitacao*> _solicitacoes_pendentes;
59
60         int _vagas_disponiveis;
61
62         bool _apenas_mulheres;
63
64         StatusCarona _status;
65
66         TipoCarona _tipo;
67     public:
68         Carona(std::string origem_nome, std::string destino_nome, Zona origem_zona,
69             Zona destino_zona, UFMGPosicao ufmg_posicao, std::string data, Usuario* motorista,
70             Veiculo* veiculo_usado, bool apenas_mulheres, TipoCarona tipo);
71
72         static int gerar_proximo_id();
73
74         int get_id() const;
```

```

154
159     Usuario* get_motorista() const;
160
165     Veiculo* get_veiculo_usado() const;
166
171     const std::string& get_origem() const;
172
177     const std::string& get_destino() const;
178
183     Zona get_origem_zona() const;
184
189     Zona get_destino_zona() const;
190
195     UFMGPosicao get_ufmg_posicao() const;
196
201     StatusCarona get_status() const;
202
207     void set_status(StatusCarona novo_status);
208
213     const std::string& get_data_hora() const;
214
219     int get_vagas_disponiveis() const;
220
225     bool get_apenas_mulheres() const;
226
230     void exibir_info() const;
231
235     void exibir_info_detalhada() const;
236
241     void adicionar_solicitacao(Solicitacao* solicitacao);
242
247     const std::vector<Solicitacao*>& get_solicitacoes_pendentes() const;
248
253     bool tem_solicitacoes_pendentes() const;
254
259     void adicionar_passageiro(Usuario* passageiro);
260
265     void remover_passageiro(Usuario* passageiro);
266 };
267 }
268 #endif

```

CaronaFactory.hpp

```
1
2
3
4
5 #ifndef CARONA_FACTORY_HPP
6 #define CARONA_FACTORY_HPP
7
8 #include "Carona.hpp"
9 #include "Zona.hpp"
10 #include <string>
11
12 namespace ufmg_carona {
13
14     // Forward declarations para evitar inclusões desnecessárias.
15     class Usuario;
16     class Veiculo;
17
18
19
20
21
22
23
24
25     class CaronaFactory {
26     public:
27
28
29
30
31
32
33
34
35         static Carona criar_carona(std::string origem_nome, std::string destino_nome,
36 Zona origem_zona, Zona destino_zona, UFMGPosicao ufmg_posicao, std::string data, Usuario*
37 motorista, Veiculo* veiculo_usado, bool apenas_mulheres, TipoCarona tipo);
38
39     };
40 }
41 #endif
```

Referência ao ficheiro include/Excecoes.hpp

Define a hierarquia de exceções customizadas para a aplicação de caronas. Centralizar as exceções em um único arquivo promove um tratamento de erros consistente e padronizado em todo o sistema.

```
#include <stdexcept>
#include <string>
```

Componentes

```
class ufmg_carona::AppExcecaoClasse base para todas as exceções específicas da aplicação.
class ufmg_carona::AutenticacaoFalhouExceptionExceção lançada quando uma tentativa de login falha.
class ufmg_carona::CaronaLotadaExceptionExceção lançada ao tentar solicitar participação em uma carona sem vagas.
class ufmg_carona::ComandoInvalidoExceptionExceção lançada quando o sistema recebe um comando de usuário não reconhecido.
```

Descrição detalhada

Define a hierarquia de exceções customizadas para a aplicação de caronas. Centralizar as exceções em um único arquivo promove um tratamento de erros consistente e padronizado em todo o sistema.

Excecoes.hpp

Ir para a documentação deste ficheiro.

```
1
2
3
4
5
6
7 #ifndef EXCECOES_HPP
8 #define EXCECOES_HPP
9
10 #include <stdexcept>
11 #include <string>
12
13 namespace ufmg_carona {
14
15     class AppExcecao : public std::runtime_error {
16     public:
17         explicit AppExcecao(const std::string& message) : std::runtime_error(message)
18     {}
19     };
20
21     class AutenticacaoFalhouException : public AppExcecao {
22     public:
23         AutenticacaoFalhouException() : AppExcecao("CPF ou senha invalidos.") {}
24     };
25
26     class CaronaLotadaException : public AppExcecao {
27     public:
28         CaronaLotadaException() : AppExcecao("A carona selecionada nao possui mais
29 vagas disponiveis.") {}
30     };
31
32     class ComandoInvalidoException : public AppExcecao {
33     public:
34         ComandoInvalidoException(const std::string& comando) : AppExcecao("Comando '"
35 + comando + "' nao reconhecido.") {}
36     };
37 }
38
39 #endif
```

Referência ao ficheiro include/Genero.hpp

Define o enumerador Genero para representar as opções de identidade de gênero dos usuários.

Enumerações

- enum class **ufmg_carona::Genero** { **MASCULINO**, **FEMININO**, **OUTRO**, **PREFIRO_NAO_INFORMAR** }

Representa as opções de identidade de gênero para um perfil de usuário.

Descrição detalhada

Define o enumerador Genero para representar as opções de identidade de gênero dos usuários.

Documentação dos valores da enumeração

enum class ufmg_carona::Genero [strong]

Representa as opções de identidade de gênero para um perfil de usuário.

Nota

O uso de 'enum class' garante segurança de tipo (type-safety), evitando conversões implícitas para inteiros.

Valores de enumerações:

MASCULINO	Representa o gênero masculino.
FEMININO	Representa o gênero feminino.
OUTRO	Para identidades de gênero não-binárias ou outras especificações.
PREFIRO_NAO_INFORMAR	Opção para o usuário que não deseja fornecer esta informação.

Genero.hpp

Ir para a documentação deste ficheiro.

```
1
5 #ifndef GENERO_HPP
6 #define GENERO_HPP
7
8 namespace ufmg_carona {
9
15     enum class Genero {
16         MASCULINO,
17         FEMININO,
18         OUTRO,
19         PREFIRO_NAO_INFORMAR
20     };
21 }
22
23 #endif
```

Referência ao ficheiro include/Motorista.hpp

Define a classe Motorista, uma especialização da classe Usuario que representa um motorista no sistema.

```
#include "Usuario.hpp"
#include "Veiculo.hpp"
#include <vector>
```

Componentes

`class ufmg_carona::Motorista` *Representa um usuário que também pode atuar como motorista, oferecendo caronas.*

Descrição detalhada

Define a classe Motorista, uma especialização da classe Usuario que representa um motorista no sistema.

Motorista.hpp

Ir para a documentação deste ficheiro.

```
1
2
3
4
5 #ifndef MOTORISTA_HPP
6 #define MOTORISTA_HPP
7
8 #include "Usuario.hpp"
9 #include "Veiculo.hpp"
10 #include <vector>
11
12 namespace ufmg_carona {
13
14     class Motorista : public Usuario {
15     private:
16         std::vector<Veiculo*>  veiculos;
17
18         std::string _cnh_numero;
19
20     public:
21         Motorista(std::string nome, std::string cpf, std::string telefone, std::string
data_nascimento,
22                 std::string email, std::string senha, Genero genero, std::string
vinculo_tipo,
23                 std::string detalhe_vinculo, std::string cnh_numero);
24
25         ~Motorista() override;
26
27         void imprimir_perfil() const override;
28
29         void adicionar_veiculo(Veiculo* veiculo);
30
31         bool is_motorista() const override;
32
33         const std::string& get_cnh_numero() const;
34
35         const std::vector<Veiculo*>& get_veiculos() const;
36
37         Veiculo* buscar_veiculo_por_placa(const std::string& placa) const;
38
39         Veiculo* buscar_veiculo_por_indice(size_t indice) const;
40
41         bool remover_veiculo(const std::string& placa);
42     };
43 }
44
45 #endif
```

Referência ao ficheiro include/Notificacao.hpp

Define a classe Notificacao, uma entidade simples para representar mensagens enviadas aos usuários.
`#include <string>`

Componentes

class ufmg_carona::Notificacao Encapsula uma mensagem e seu estado (lida/não lida) para comunicação com o usuário. Esta classe é usada para informar os usuários sobre eventos importantes, como a aceitação de uma solicitação de carona ou uma nova avaliação recebida.

Descrição detalhada

Define a classe Notificacao, uma entidade simples para representar mensagens enviadas aos usuários.

Notificacao.hpp

Ir para a documentação deste ficheiro.

```
1
5 #ifndef NOTIFICACAO_HPP
6 #define NOTIFICACAO_HPP
7
8 #include <string>
9
10 namespace ufmg carona {
11
12     class Notificacao {
13     private:
14         std::string _mensagem;
15
16         bool lida;
17
18     public:
19         Notificacao(std::string mensagem);
20
21         void marcar como lida();
22
23         const std::string& get_mensagem() const;
24
25         bool is_lida() const;
26     };
27 }
28
29 #endif
```

Referência ao ficheiro include/Sistema.hpp

Define a classe Sistema, o orquestrador central de toda a aplicação de caronas.

```
#include <vector>
#include <string>
#include <tuple>
#include <fstream>
#include <ctime>
#include <map>
#include "Usuario.hpp"
#include "Motorista.hpp"
#include "Carona.hpp"
#include "Solicitacao.hpp"
#include "Genero.hpp"
#include "Veiculo.hpp"
#include "Avaliacao.hpp"
#include "Zona.hpp"
```

Componentes

class ufmg_carona::SistemaA classe principal que gerencia o estado, os dados e os fluxos de interação do usuário. Atua como um orquestrador para toda a aplicação, inicializando subsistemas, controlando o loop principal e garantindo a persistência dos dados.

Descrição detalhada

Define a classe Sistema, o orquestrador central de toda a aplicação de caronas.

Sistema.hpp

Ir para a documentação deste ficheiro.

```
1
2
3
4
5 #ifndef SISTEMA_HPP
6 #define SISTEMA_HPP
7
8 #include <vector>
9 #include <string>
10 #include <tuple>
11 #include <fstream>
12 #include <ctime>
13 #include <map>
14
15 #include "Usuario.hpp"
16 #include "Motorista.hpp"
17 #include "Carona.hpp"
18 #include "Solicitacao.hpp"
19 #include "Genero.hpp"
20 #include "Veiculo.hpp"
21 #include "Avaliacao.hpp"
22 #include "Zona.hpp"
23
24 namespace ufmg_carona {
25
26     class Sistema {
27     private:
28
29         std::vector<Usuario*> _usuarios;
30
31         std::vector<Carona> _caronas;
32
33         std::vector<Solicitacao*> _solicitacoes;
34
35         std::vector<Avaliacao*> _avaliacoes_globais;
36
37         Usuario* _usuario_logado;
38
39         std::map<int, Zona> _int_para_zona;
40         std::map<Zona, std::string> _zona_para_string;
41         std::map<int, UFMGPosicao> _int_para_ufmg_posicao;
42         std::map<UFMGPosicao, std::string> _ufmg_posicao_para_string;
43
44         void carregar_dados_iniciais();
45         void salvar_dados_usuarios();
46         void salvar_dados_veiculos();
47         void salvar_dados_caronas();
48         void carregar_dados_caronas();
49         void carregar_dados_solicitacoes();
50         void salvar_dados_solicitacoes();
51         void salvar_dados_avaliacoes();
52         void carregar_dados_avaliacoes();
53
54         Usuario* buscar_usuario_por_cpf(const std::string& cpf);
55         Carona* buscar_carona_por_id(int id);
56         Veiculo* buscar_veiculo_por_placa_motorista(Motorista* motorista, const
std::string& placa);
57         std::tuple<bool, std::string, std::string, std::string, std::string>
58         buscar_dados_ufmg_por_cpf(const std::string& cpf_buscado);
59
60         void exibir_menu_inicial_nao_logado();
61         void exibir_menu_logado();
62         void exibir_menu_passageiro();
63         void exibir_menu_motorista();
64         void exibir_menu();
65
66         void processar_comando_logado(const std::string& comando_str);
67
68     };
69
70 }
```

```

131
132 void fluxo_cadastro();
133 void fluxo_login();
134 void fluxo_logout();
135 void fluxo_oferecer_carona();
136 void fluxo_solicitar_carona();
137 void fluxo_status_caronas();
138 void fluxo_cadastrar_veiculo();
139 void fluxo_editar_perfil();
140 void fluxo_passageiro_menu();
141 void fluxo_motorista_menu();
142 void fluxo_editar_perfil_ou_veiculos();
143 void fluxo_tornar_motorista();
144 void fluxo_gerenciar_veiculos();
145 void fluxo_editar_veiculo(Motorista* motorista);
146 void fluxo_excluir_veiculo(Motorista* motorista);
147 void fluxo_gerenciar_caronas();
148 void fluxo_solicitacoes_pendentes_motorista();
149 void fluxo_minhas_caronas(Motorista* motorista_logado);
150 void fluxo_avaliacoes();
151 void fluxo_avaliar_carona_passageiro();
152 void fluxo_avaliar_passageiros_motorista();
153 void exibir_minhas_avaliacoes_recebidas();
154 void exibir_avaliacoes_que_fiz();
155
156
157 void enviar_notificacao(Usuario* usuario, const std::string& mensagem, bool
158 enviar_para_motorista = true);
159 bool pode_solicitar_carona(Usuario* passageiro, const Carona& carona);
160 void cancelar_outras_solicitacoes_passageiro(Usuario* passageiro, const
161 Carona& carona_aceita);
162 void finalizar_carona_completa(Carona* carona_para_finalizar);
163 void cancelar_carona_completa(Carona* carona_para_cancelar);
164 void remover_caronas_passadas();
165
166 int coletar_int_input(const std::string& prompt, int min_val, int max_val);
167 std::string coletar_string_input(const std::string& prompt);
168 Zona coletar_zona_input(const std::string& prompt);
169 UFMGPosicao coletar_ufmg_posicao_input(const std::string& prompt);
170 std::string zona_to_string(Zona z) const;
171 Zona string_to_zona(const std::string& s) const;
172 std::string ufmg_posicao_to_string(UFMGPosicao up) const;
173 UFMGPosicao string_to_ufmg_posicao(const std::string& s) const;
174 std::tm parse_datetime_string(const std::string& dt_str) const;
175 std::string get_current_datetime_string() const;
176 bool is_datetime_in_past(const std::string& dt_str) const;
177
178 public:
179     Sistema();
180
181     ~Sistema();
182
183     void executar();
184 };
185 }
186 #endif

```

Referência ao ficheiro include/Solicitacao.hpp

Define a classe Solicitacao, que modela a requisição de um passageiro para entrar em uma carona.

```
#include <string>
```

Componentes

class **ufmg_carona::Solicitacao** *Modela uma solicitação de um passageiro para participar de uma carona. Esta classe encapsula todo o fluxo de negociação, incluindo propostas de locais e o estado final da solicitação, além de rastrear o status das avaliações pós-viagem.*

Enumerações

- enum class **ufmg_carona::StatusSolicitacao** { **PENDENTE, ACEITA, RECUSADA, AGUARDANDO_RESPOSTA_PASSAGEIRO, RECUSADA_PROPOSTA_MOTORISTA** }
Define o ciclo de vida e os estados possíveis de uma solicitação de carona. Representa a máquina de estados do processo de negociação entre passageiro e motorista.

Descrição detalhada

Define a classe Solicitacao, que modela a requisição de um passageiro para entrar em uma carona.

Documentação dos valores da enumeração

enum class ufmɡ_carona::StatusSolicitacao [strong]

Define o ciclo de vida e os estados possíveis de uma solicitação de carona. Representa a máquina de estados do processo de negociação entre passageiro e motorista.

Valores de enumerações:

PENDENTE	A solicitação foi enviada e aguarda ação do motorista.
ACEITA	O motorista aceitou o passageiro na carona.
RECUSADA	O motorista recusou a solicitação.
AGUARDANDO_RESPOSTA_PASSAGEIRO	O motorista fez uma contraproposta de local e aguarda a resposta do passageiro.
RECUSADA_PROPOSTA_MOTORISTA	O passageiro recusou a contraproposta do motorista.

Solicitacao.hpp

Ir para a documentação deste ficheiro.

```
1
2
3
4
5 #ifndef SOLICITACAO_HPP
6 #define SOLICITACAO_HPP
7
8 #include <string>
9
10 namespace ufmg carona {
11
12     // Forward declarations para evitar dependências circulares.
13     class Usuario;
14     class Carona;
15
16
17
18
19
20     enum class StatusSolicitacao {
21         PENDENTE,
22         ACEITA,
23         RECUSADA,
24         AGUARDANDO_RESPOSTA_PASSAGEIRO,
25         RECUSADA_PROPOSTA_MOTORISTA
26     };
27
28
29
30
31
32
33
34     class Solicitacao {
35     private:
36         Usuario* _passageiro;
37
38
39
40         Carona* _carona_alvo;
41
42
43
44         StatusSolicitacao _status;
45
46
47         std::string _local_embarque_passageiro;
48
49         std::string _local_desembarque_passageiro;
50
51
52         std::string _local_embarque_motorista_proposto;
53
54         std::string _local_desembarque_motorista_proposto;
55
56         bool _passageiro_avaliou_motorista;
57
58         bool _motorista_avaliou_passageiro;
59
60     public:
61         Solicitacao(Usuario* passageiro, Carona* carona, std::string
62 local_embarque_p, std::string local_desembarque_p);
63
64
65
66
67         void aceitar();
68         void recusar();
69         void propor_locais_motorista(std::string local_embarque_m, std::string
64 local_desembarque_m);
65         void aceitar_proposta_motorista();
66         void recusar_proposta_motorista();
67
68
69
70
71         void set_status(StatusSolicitacao novo_status);
72         void set_carona(Carona* new_carona_ptr);
73
74
75         Usuario* get_passageiro() const;
76         Carona* get_carona() const;
77         StatusSolicitacao get_status() const;
78         std::string get_status_string() const;
79         const std::string& get_local_embarque_passageiro() const;
80         const std::string& get_local_desembarque_passageiro() const;
81         const std::string& get_local_embarque_motorista_proposto() const;
82         const std::string& get_local_desembarque_motorista_proposto() const;
83
84
85
86
87         void exibir_info() const;
88         void exibir_para_motorista() const;
89
90
91         bool get_passageiro_avaliou_motorista() const;
```



```
139     void set_passageiro_avalizou_motorista(bool avaliou);
140     bool get_motorista_avalizou_passageiro() const;
141     void set_motorista_avalizou_passageiro(bool avaliou);
142 };
143 }
144 }
145 #endif
```

Referência ao ficheiro include/Usuario.hpp

Define a classe Usuario, a classe base polimórfica para todos os tipos de usuários no sistema.

```
#include <string>
#include <vector>
#include "Notificacao.hpp"
#include "Avaliacao.hpp"
#include "Genero.hpp"
```

Componentes

*class ufmg_carona::UsuarioClasse base que encapsula os dados e comportamentos comuns a todos os usuários. Serve como a fundação para o polimorfismo no sistema, permitindo que diferentes tipos de usuários (como **Motorista**) sejam tratados de forma uniforme.*

Descrição detalhada

Define a classe Usuario, a classe base polimórfica para todos os tipos de usuários no sistema.

Usuario.hpp

Ir para a documentação deste ficheiro.

```
1
5 #ifndef USUARIO_HPP
6 #define USUARIO_HPP
7
8 #include <string>
9 #include <vector>
10 #include "Notificacao.hpp"
11 #include "Avaliacao.hpp"
12 #include "Genero.hpp"
13
14 namespace ufmg_carona {
15
16     class Usuario {
17     protected:
18         std::string _nome, _cpf, _email, _senha;
19         std::string _telefone;
20         std::string _data_nascimento;
21         Genero genero;
22         std::vector<Avaliacao*> _avaliacoes_recebidas;
23         std::vector<Notificacao> _notificacoes;
24         std::string _vinculo_tipo;
25         std::string _detalhe_vinculo;
26
27     public:
28         Usuario(std::string nome, std::string cpf, std::string telefone, std::string
data_nascimento, std::string email, std::string senha, Genero genero, std::string
vinculo_tipo, std::string detalhe_vinculo);
29
30         virtual ~Usuario();
31
32         std::string get_vinculo() const;
33         std::string get_vinculo_raw() const;
34         std::string get_detalhe_vinculo() const;
35         const std::string& get_email() const;
36         const std::string& get_senha() const;
37         Genero get_genero() const;
38         const std::string& get_cpf() const;
39         const std::string& get_nome() const;
40         const std::string& get_telefone() const;
41         const std::string& get_data_nascimento() const;
42
43         bool verificar_senha(const std::string& senha) const;
44
45         virtual bool is_motorista() const;
46
47         double get_media_avaliacoes() const;
48         std::string get_medalha() const;
49         void adicionar_avaliacao_recebida(Avaliacao* avaliacao);
50
51         virtual void imprimir_perfil() const;
52         void adicionar_notificacao(const Notificacao& notificacao);
53         const std::vector<Notificacao>& get_notificacoes() const;
54         std::vector<Notificacao>& get_notificacoes_mutavel();
55
56         void set_email(const std::string& email);
57         void set_telefone(const std::string& telefone);
58         void set_senha(const std::string& senha);
59         void set_genero(Genero genero);
60     };
61 }
62 #endif
```

Referência ao ficheiro include/Veiculo.hpp

Define a classe Veiculo, que encapsula os dados de um veículo cadastrado no sistema.

```
#include <string>
```

Componentes

`class ufmg_carona::Veiculo` Modela um veículo, contendo todas as suas informações relevantes. Esta classe atua como uma estrutura de dados para armazenar os detalhes de um veículo que pode ser utilizado em uma carona.

Descrição detalhada

Define a classe Veiculo, que encapsula os dados de um veículo cadastrado no sistema.

Veiculo.hpp

Ir para a documentação deste ficheiro.

```
1
5 #ifndef VEICULO_HPP
6 #define VEICULO_HPP
7
8 #include <string>
9
10 namespace ufmg carona {
11
12     class Veiculo {
13     private:
14         std::string _placa;
15         std::string _marca;
16         std::string _modelo;
17         std::string _cor;
18
19         int _total_de_lugares;
20
21     public:
22         Veiculo(const std::string& placa, const std::string& marca, const std::string&
23 modelo, const std::string& cor, int lugares);
24
25         Veiculo();
26
27         int get_lugares() const;
28
29         const std::string& get_placa() const;
30         const std::string& get_marca() const;
31         const std::string& get_modelo() const;
32         const std::string& get_cor() const;
33
34         void set_marca(const std::string& marca);
35         void set_modelo(const std::string& modelo);
36         void set_cor(const std::string& cor);
37         void set_lugares(int lugares);
38
39         void exhibir_info() const;
40     };
41 }
42
43 #endif
```

Referência ao ficheiro include/Zona.hpp

Define os enumeradores geográficos utilizados para classificar as rotas das caronas.

Enumerações

- enum class **ufmg_carona::Zona** { **PAMPULHA**, **CENTRO_SUL**, **NOROESTE**, **LESTE**, **OESTE**, **NORTE**, **VENDA_NOVA**, **BARREIRO** }
Representa as zonas geográficas (regionais) de Belo Horizonte. Utilizado para agrupar e buscar caronas com base na localização de origem ou destino na cidade.
- enum class **ufmg_carona::UFMGPosicao** { **ORIGEM**, **DESTINO** }
Especifica se o campus da UFMG é o ponto de partida ou de chegada de uma carona. Facilita a busca por rotas que começam ou terminam na universidade.

Descrição detalhada

Define os enumeradores geográficos utilizados para classificar as rotas das caronas.

Documentação dos valores da enumeração

enum class ufm_g_carona::UFMGPosicao [strong]

Especifica se o campus da UFMG é o ponto de partida ou de chegada de uma carona. Facilita a busca por rotas que começam ou terminam na universidade.

Valores de enumerações:

ORIGEM	A carona se inicia na UFMG e tem como destino uma zona da cidade.
DESTINO	A carona se inicia em uma zona da cidade e tem como destino a UFMG.

Zona.hpp

Ir para a documentação deste ficheiro.

```
1
5 #ifndef ZONA_HPP
6 #define ZONA_HPP
7
8 namespace ufmg_carona {
9
15     enum class Zona {
16         PAMPULHA,
17         CENTRO_SUL,
18         NOROESTE,
19         LESTE,
20         OESTE,
21         NORTE,
22         VENDA_NOVA,
23         BARREIRO
24     };
25
30     enum class UFMGPosicao {
31         ORIGEM,
32         DESTINO
33     };
34 }
35
36 #endif
```

Índice

INDEX